

Preliminaries: machine & deep learning, feature extraction and graphs

Preliminaries

Reminder: Supervised regression (linear)

Single regression model

Multi-regression models

$$l(w) = \frac{1}{n} \sum_{i=1}^n (\tilde{x}_i^T w - y_i)^2 \quad (L^2 \text{ loss})$$

fit of sample i

$\tilde{X} \in \mathbb{R}^{n \times (d+1)}$ $w \in \mathbb{R}^{d+1}$ $\hat{y} \in \mathbb{R}^n$ $y \in \mathbb{R}^n$

A single layer linear network with a single output

$$l(W) = \|\tilde{X}W - Y\|_2^2 = \frac{1}{n} \sum_{c=1}^C (\tilde{X}w_c - y_c)^2$$

fit of sample i

$\tilde{X} \in \mathbb{R}^{n \times (d+1)}$ $W \in \mathbb{R}^{(d+1) \times C}$ $\hat{Y} \in \mathbb{R}^{n \times C}$ $Y \in \mathbb{R}^{n \times C}$

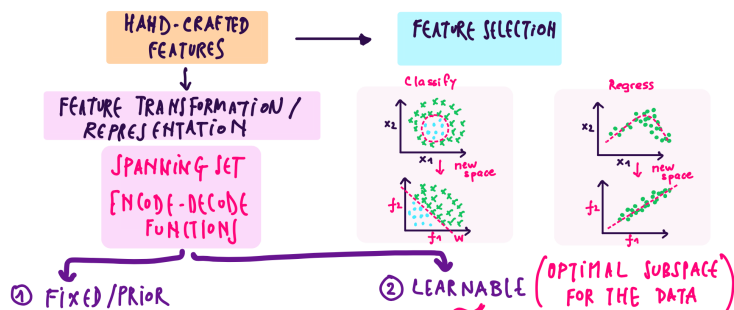
A single layer linear network with a multiple outputs

- Intuitive Deep Learning Series
1. Intuitive Deep Learning 1.1: Supervised linear and non-linear models
 2. Intuitive Deep Learning 1.2: Supervised linear and non-linear models
 3. Intuitive Deep Learning 1.3: Supervised linear and non-linear models
 4. Intuitive Deep Learning 1.4: Sparring sets & matrix factorization
 5. Intuitive Deep Learning 1.5: Sparring sets & matrix factorization
 6. Intuitive Deep Learning 1.6: Generalized matrix factorization
 7. Deep Learning Matrix Factorization: Sparring sets
 8. Deep Learning Matrix Factorization: Sparring sets

DL₁

CLASSIC FEATURE REPRESENTATION METHODS

ML

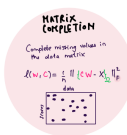


Aim

Transform complex features into a space where linear models can work well in the final representation space.

MATRIX FACTORIZATION

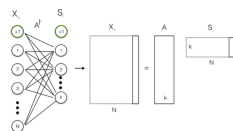
$CW \approx X$: we factor the matrix X into a product of two matrices C & W .



A Deep Non-Negative Matrix Factorization Neural Network

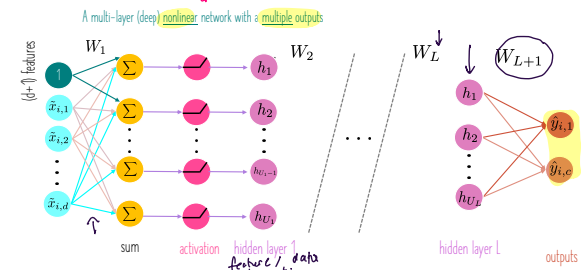
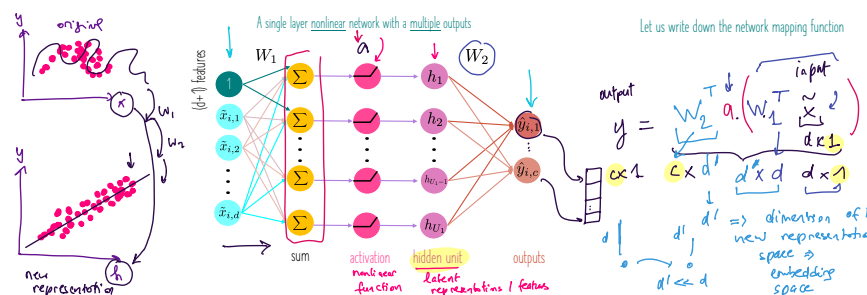
Jonathan Pflaum - Blake Hout

Theorem 1 A standard deep NMF layer can be written as a multi-layered deep non-negative matrix factorization (Deep NMF) architecture



DL

COMPOSITION & BACKPROPAGATION



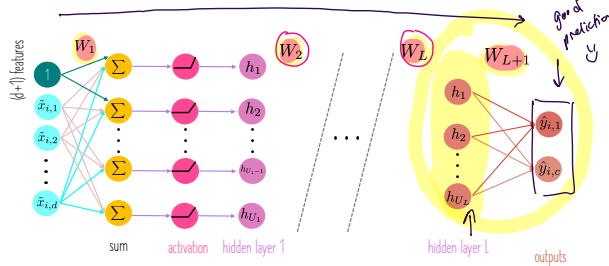
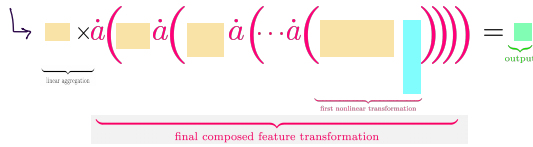
$$w_{L+1}^T \circ a(w_L^T \circ a(w_{L-1}^T \circ a(w_{L-2}^T \circ \dots \circ a(w_1^T x)))) = \hat{y}_{L+1}$$

linear aggregate

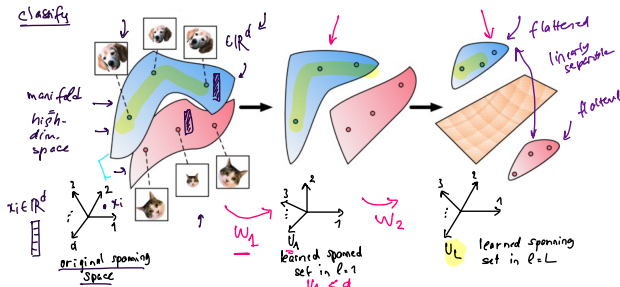
final composed feature transformation

Feature embedding

Feature embedding in deep learning and manifold learning

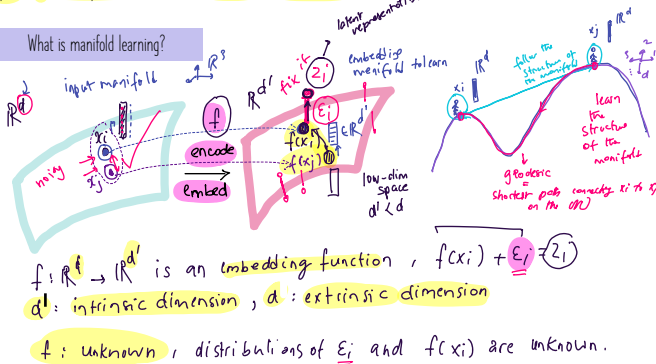


With higher computational budget we achieve the Da Vinci principle: "Simplicity is the Ultimate Sophistication"



The power of DL models lies in learning how to embed complex features sequentially into manifolds (subspaces to learn) that decrease in complexity until 'finding' an ultimate low-dimensional representation space (a much simpler manifold) where linear models can work well.

What is manifold learning?



- Underlying reductionist hypothesis: neighborhood preservation across the original and embedding spaces. ??
- The neighborhood in the original domain may be highly noisy (outliers in the local neighborhood of a sample).

The evolving landscape of feature embedding

ML problem formalization

$$l(w) = \frac{1}{n} \sum_{i=1}^n (\tilde{x}_i^T w - y_i)^2$$

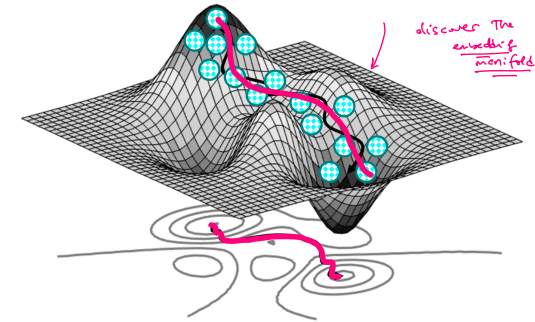
$$\begin{matrix} (d+1) \text{ features} \\ n \text{ samples} \end{matrix} \times \begin{matrix} (d+1) \text{ features} \\ n \text{ samples} \end{matrix} = \begin{matrix} n \text{ samples} \\ n \text{ samples} \end{matrix} \approx \begin{matrix} n \text{ samples} \\ n \text{ samples} \end{matrix}$$

$$\tilde{X} \in \mathbb{R}^{n \times (d+1)} \quad w \in \mathbb{R}^{d+1} \quad \hat{y} \in \mathbb{R}^n \quad y \in \mathbb{R}^n$$

Underlying reductionist assumption

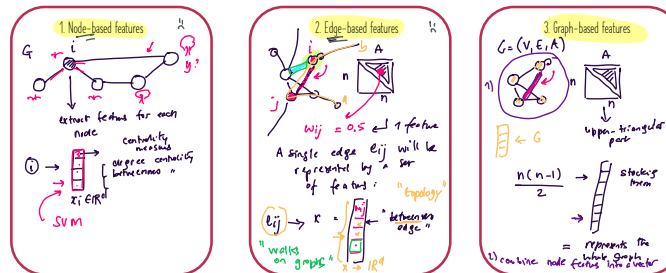
We learn a shared mapping w across all training samples. However, our training samples are not allowed to directly interact.

Individualistic points -- data samples live "independently" on a high-dimensional manifold.

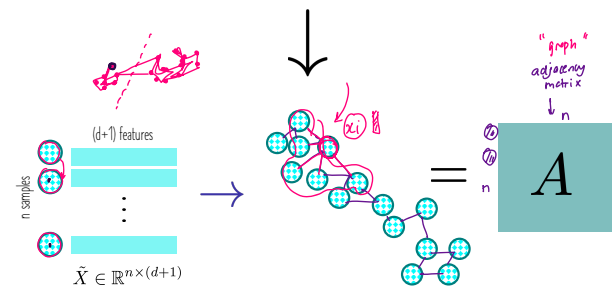
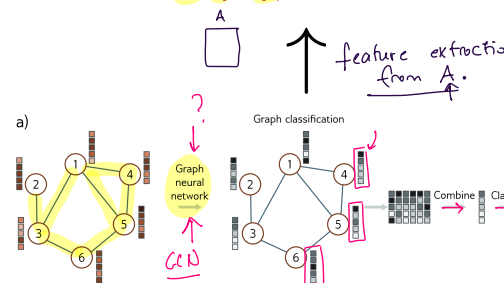


Even in manifold learning, we learn how to calculate the similarity between points but not how to connect them. We are one step away from that!

- Cannot utilize node, edge and graph features.
- Such approaches cannot obtain embeddings for nodes not in the training set. If we add a node, we need to recalculate all node feature representations (embeddings).

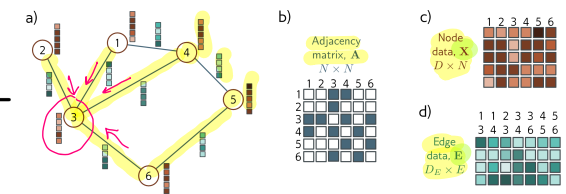


Classical node, edge and graph-based feature extraction methods



Use a graph to model and capture the relationships between pairs of data points. Each sample interacts with all other samples.

Collectivist points -- data samples co-interact and co-learn together via a graph structure (A)



We can also add edge features. Remember you've got a graph to learn on!

Graph Convolutional Networks (UDL-13)

Introducing layer-wise non-linearity and biases into GCN

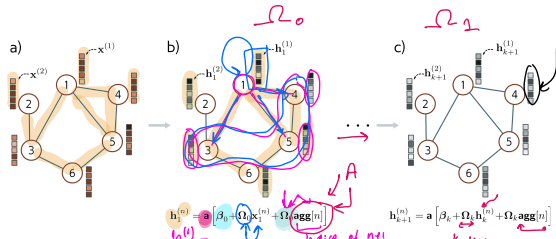


Figure: Simple Graph CNN layer. (a) Input graph consists of structure (embodied in graph adjacency matrix A , not shown) and node embeddings (stored in columns of X). (b) Each node in the first hidden layer is updated by (i) aggregating the neighboring nodes to form a single vector, (ii) applying a linear transformation Ω_0 to the aggregated nodes, (iii) applying the same linear transformation Ω_0 to the original node, (iv) adding these together with a bias β_0 , and finally (v) applying a nonlinear activation function $a(\cdot)$ like a ReLU. This process is repeated at subsequent layers (but with different parameters for each layer) until we produce the final embeddings at the end of the network.

Node embedding update rule

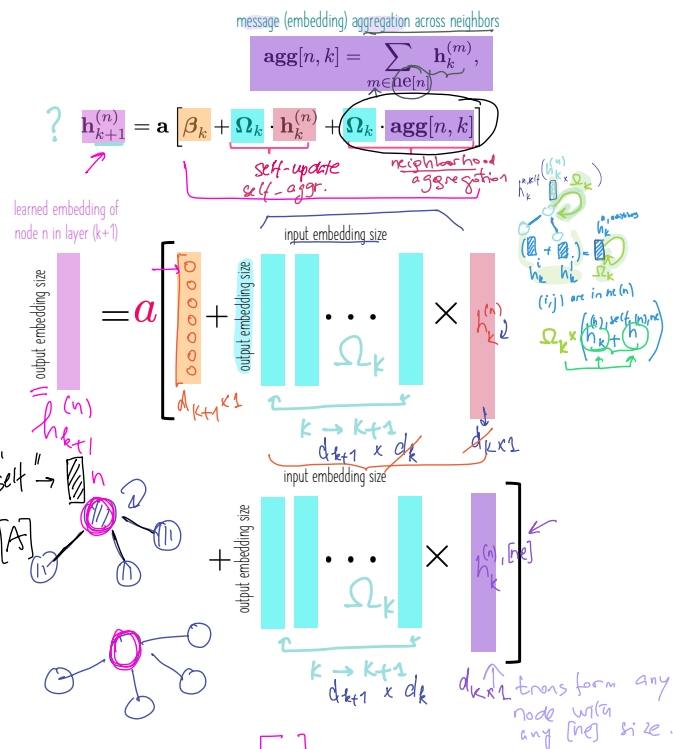
$$A = \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix}$$

message (embedding) aggregation across neighbors

$$\text{agg}[n, k] = \sum_{m \in \text{ne}(n)} h_k^{(m)}$$

$$h_{k+1}^{(n)} = a[\beta_k + \Omega_k \cdot h_k^{(n)} + \Omega_k \cdot \text{agg}[n, k]]$$

self-update self-aggr. neighborhood aggregation



Update rule for all nodes

$$H_{k+1} = a[\beta_k \mathbf{1}^T + \Omega_k H_k + \Omega_k H_k A]$$

$$H_{k+1} = a[\beta_k \mathbf{1}^T + \Omega_k H_k (A + \mathbf{I})]$$

aggr neighbors nodes for each node

ReLU

Example: graph classification

Graph classification

Combine

Classify

Class 1

Class 2

Class 3

Classifier MLP

Deep (sequential) node representation learning

Network output function (output prediction)

$$f(X, A, \Phi) = \text{sig}[\beta_K + \omega_K H_K \mathbf{1}/N]$$

$$y = \text{sig}[\beta_K + \omega_K H_K \mathbf{1}/N]$$

Section 3 of DGL 1.3

good representation of all samples

nodes

class

1

0.9

0.9

Going deeper by adding more layers

How to combine the current node and aggregated neighbors?

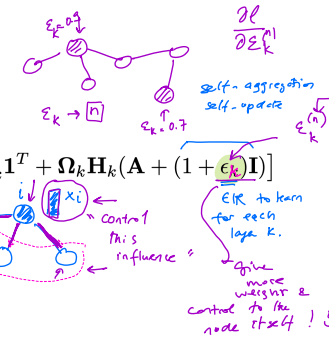
Simply sum them up

$$H_{k+1} = a[\beta_k \mathbf{1}^T + \Omega_k H_k (A + \mathbf{I})]$$

self-embedding update = self-aggregation

$\hat{A} \rightarrow$ vanilla GCN

Add a diagonal enhancement



$$H_{k+1} = a[\beta_k \mathbf{1}^T + \Omega_k H_k (A + (1 + \epsilon_k) \mathbf{I})]$$

Going deeper with more layers

Treat the current node differently from the aggregated neighbors by applying a different linear transform Ψ to the current node:

$$H_{k+1} = a[\beta_k \mathbf{1}^T + \Psi_k H_k + \Omega_k H_k A]$$

$$= a[\beta_k \mathbf{1}^T + [\Omega_k \quad \Psi_k] \begin{bmatrix} H_k A \\ H_k \end{bmatrix}]$$

$$= a[\beta_k \mathbf{1}^T + \tilde{\Omega}_k \begin{bmatrix} H_k A \\ H_k \end{bmatrix}]$$

self-transformation via Ψ_k

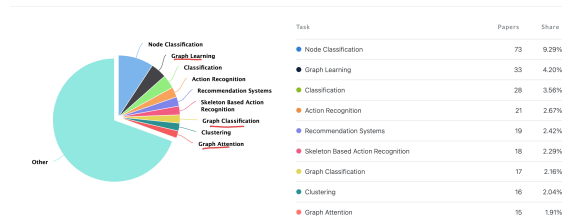
neighborhood transformation via Ω_k

Graph Convolutional Networks (UDL-13)

GCNs are hot topics!

Papers with code

Tasks



Usage Over Time



⚠ This feature is experimental, we are continuously improving our matching algorithm.

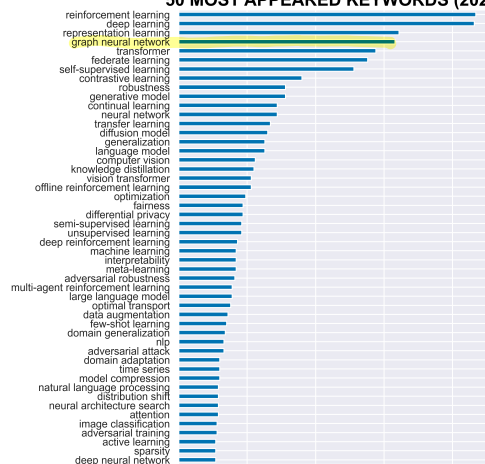
<https://paperswithcode.com/method/gcn>

ICLR 2023 open review data

• Top-10 Ranking between 2022 and 2023 (full list please refer to [keywords.md](#))

Keyword	2022	2023
reinforcement learning	1	1
deep learning	2	2
representation learning	4	3
graph neural network	3	4
transformer	5	5
federate learning	7	6
self-supervised learning	6	7
contrastive learning	10	8
robustness	9	9
generative model	8	10

50 MOST APPEARED KEYWORDS (2023)



<https://github.com/Edvardos/ICLR2023-OpenReviewData/blob/master/keywords.md>

Recall box: summarize what you remember from this lecture below